Estudio de Avatares

Desplegar en un servidor Ubuntu en casa, con salida a internet

Guía completa: preparar el sistema, instalar la aplicación, publicarla sin abrir puertos del router y dejarla vigilada.

Ubuntu Server 22.04 / 24.04 · Python 3.11 · Gunicorn + Nginx · Cloudflare Tunnel

> **Antes de nada:** esta aplicación consume **un núcleo entero durante varios minutos** por cada video. No es una web normal. Planifica el hardware y los límites pensando en eso, no en visitas por segundo.

1 Qué necesitas y qué decidir

Hardware

Lo medido en este proyecto: **273 ms por fotograma** con dos avatares que hablan, **196 MB** de memoria en el pico. La memoria no es problema; la CPU lo es todo.

NÚCLEOS DEL EQUIPO	RENDERS A LA VEZ	VIDEOS DE 30 S POR HORA
2	1	≈ 14
4	2 *(recomendado)*	≈ 29
8	4	≈ 58

Deja siempre un núcleo libre para el sistema y para Nginx. La aplicación trae un tope de **2 renders simultáneos**; se cambia en `motor/seguridad.py`, constante `MAX_RENDERS`.

Disco: unos 3 MB por video. Se borran solos a las 12 horas.

La decisión importante: cómo sales a internet

A · Túnel de Cloudflare

El servidor abre una conexión **saliente** y Cloudflare te da un dominio con HTTPS. **No abres ningún puerto del router.** Es gratis y es lo recomendable en una casa.

B · IP pública y puertos

Abres el 80 y el 443 en el router hacia el servidor. Necesitas IP pública (no CG-NAT), DNS dinámico si cambia, y te encargas tú del certificado.

Si tu operador usa CG-NAT, la opción B no funciona.

Compruébalo: si la IP que ves en `ifconfig.me` no coincide con la WAN de tu router, estás tras CG-NAT. En ese caso, túnel sí o sí.

2 Preparar Ubuntu

Desde una sesión con `sudo`:

```
# Sistema al día
sudo apt update && sudo apt upgrade -y

# Todo lo que hace falta
sudo apt install -y ffmpeg python3-venv python3-pip git nginx \
                    fonts-dejavu-core ufw

# Comprobar que FFmpeg quedó en el PATH
ffmpeg -version | head -1
```

> **Por qué `fonts-dejavu-core`**
>
> Los subtítulos necesitan una tipografía negra. En Windows se usa Segoe UI Black, que es de Microsoft y **no se puede redistribuir** en un servidor. La aplicación detecta DejaVu Sans Bold automáticamente.

Un usuario solo para esto

Nunca ejecutes la aplicación como `root` ni con tu usuario personal.

```
sudo adduser --system --group --home /opt/estudio estudio
sudo mkdir -p /opt/estudio/app
sudo chown -R estudio:estudio /opt/estudio
```

3 Instalar la aplicación

```
sudo -u estudio -H bash
cd /opt/estudio

git clone https://github.com/encrypthdd/animacion-voz-avatar.git app

python3 -m venv venv
./venv/bin/pip install --upgrade pip
./venv/bin/pip install -r app/requirements.txt
./venv/bin/pip install gunicorn

# Reconstruir lo que no se versiona (son resultados, no fuentes)
cd app/motor
../../venv/bin/python medir.py     # mide avatares/  → rigs/
../../venv/bin/python poses.py     # recorta poses   → poses/
exit
```

Gunicorn no está en `requirements.txt` a propósito

No funciona en Windows y el proyecto también se usa en local. Es una dependencia del despliegue, no de la aplicación.

Prueba rápida antes de seguir

```
cd /opt/estudio/app/motor
sudo -u estudio ESTUDIO_CLAVE=prueba \
     /opt/estudio/venv/bin/python app.py
```

Desde otra terminal del mismo servidor: `curl -I http://127.0.0.1:5000/entrar` debe responder `200`. Corta con `Ctrl+C`.

4 Configuración

Las opciones se pasan por variables de entorno, en un archivo que solo puede leer `root`:

```
sudo nano /etc/estudio.env
```

```
ESTUDIO_CLAVE=pon-aqui-una-contraseña-larga
ESTUDIO_SECRETO=# genera una con el comando de abajo
ESTUDIO_PROXY=1
ESTUDIO_HTTPS=1
```

```
# Para generar el secreto de firma de sesiones
python3 -c "import secrets; print(secrets.token_urlsafe(32))"

# Que no lo lea nadie más
sudo chmod 600 /etc/estudio.env
```

VARIABLE	QUÉ HACE
`ESTUDIO_CLAVE`	Contraseña de acceso. **Obligatoria aquí:** con un proxy delante el servidor se niega a arrancar sin ella. En local, donde solo escucha en `127.0.0.1`, no se pide nada.
`ESTUDIO_SECRETO`	Firma las cookies de sesión. Si cambia, se cierran todas las sesiones.
`ESTUDIO_PROXY`	**1 solo si hay Nginx o un túnel delante.** Hace que se lea la IP real de `X-Forwarded-For`.
`ESTUDIO_HTTPS`	1 cuando sirvas por HTTPS. Marca la cookie como `Secure`.
`ESTUDIO_FFMPEG`	Opcional. Solo si FFmpeg no está en el PATH.
`ESTUDIO_FUENTE`	Opcional. Ruta a un `.ttf` propio para los subtítulos.

Nunca pongas `ESTUDIO_PROXY=1` sin proxy delante

Ese encabezado lo escribe quien hace la petición. Si confías en él sin un proxy que lo reescriba, cualquiera cambia su IP aparente en cada petición y **los límites por IP dejan de servir para nada**.

5 Servicio del sistema

```
sudo nano /etc/systemd/system/estudio.service
```

```
[Unit]
Description=Estudio de Avatares
After=network.target

[Service]
Type=simple
User=estudio
Group=estudio
WorkingDirectory=/opt/estudio/app/motor
EnvironmentFile=/etc/estudio.env
ExecStart=/opt/estudio/venv/bin/gunicorn \
          --workers 1 --threads 8 --worker-class gthread \
          --timeout 300 --bind 127.0.0.1:5000 app:app
Restart=on-failure
RestartSec=5

# Endurecido
NoNewPrivileges=true
PrivateTmp=true
ProtectSystem=full
ProtectHome=true
ReadWritePaths=/opt/estudio/app
# Techo de CPU: 200% = dos núcleos. El equipo sigue respondiendo.
CPUQuota=200%

[Install]
WantedBy=multi-user.target
```

```
sudo systemctl daemon-reload
sudo systemctl enable --now estudio
sudo systemctl status estudio
```

Un solo `worker`, y no es negociable

El estado de los renders vive en memoria del proceso. Con dos workers, el navegador pregunta por un trabajo y puede caer en el proceso que no lo lanzó: devolvería **404 para siempre** aunque el video se esté montando bien. Los `--threads` sí son seguros porque comparten memoria.

Para ver qué pasa: `journalctl -u estudio -f`

6 Nginx por delante

```
sudo nano /etc/nginx/sites-available/estudio
```

```
# Límite por IP a nivel de red, antes de llegar a Python
limit_req_zone $binary_remote_addr zone=estudio:10m rate=60r/m;

server {
    listen 80;
    server_name avatares.tudominio.com;

    # Debe coincidir con el límite de subida de la aplicación
    client_max_body_size 40m;

    # No anunciar la versión de Nginx
    server_tokens off;

    location / {
        limit_req zone=estudio burst=30 nodelay;

        proxy_pass http://127.0.0.1:5000;
        proxy_set_header Host              $host;
        proxy_set_header X-Real-IP         $remote_addr;
        proxy_set_header X-Forwarded-For   $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;

        # Los videos se sirven por partes; no cortar
        proxy_read_timeout 300s;
        proxy_buffering off;
    }
}
```

```
sudo ln -s /etc/nginx/sites-available/estudio /etc/nginx/sites-enabled/
sudo rm -f /etc/nginx/sites-enabled/default
sudo nginx -t && sudo systemctl reload nginx
```

Dos capas de límites, a propósito

Nginx corta la avalancha barata antes de gastar un proceso de Python. La aplicación aplica además límites por tipo de acción: 6 renders cada 15 minutos por IP, 20 subidas, 8 intentos de contraseña. Están en `motor/seguridad.py`, diccionario `LIMITES`.

7 Salida a internet

Opción A · Túnel de Cloudflare *(recomendada en casa)*

Necesitas un dominio con el DNS en Cloudflare. La cuenta gratuita basta.

```
curl -L --output cloudflared.deb \
  https://github.com/cloudflare/cloudflared/releases/latest/download/cloudflared-linux-
amd64.deb
sudo dpkg -i cloudflared.deb

cloudflared tunnel login          # abre un enlace, autoriza el dominio
cloudflared tunnel create estudio

sudo nano /etc/cloudflared/config.yml
```

```
tunnel: estudio
credentials-file: /root/.cloudflared/<ID-DEL-TUNEL>.json

ingress:
  - hostname: avatares.tudominio.com
    service: http://127.0.0.1:80
  - service: http_status:404
```

```
cloudflared tunnel route dns estudio avatares.tudominio.com
sudo cloudflared service install
sudo systemctl status cloudflared
```

Ya tienes HTTPS, certificado automático y **cero puertos abiertos**.

> **El límite de 100 segundos de Cloudflare no te afecta**
>
> Cloudflare corta las peticiones largas. Un render tarda minutos, pero la aplicación es asíncrona: la petición que lanza el trabajo responde al instante con un identificador y el navegador va preguntando. Ninguna petición dura tanto.

Opción B · Puertos abiertos y certificado propio

```
# En el router: redirigir 80 y 443 a la IP del servidor
sudo certbot --nginx -d avatares.tudominio.com
sudo systemctl status certbot.timer   # la renovación es automática
```

Cortafuegos

```
sudo ufw default deny incoming
sudo ufw default allow outgoing
sudo ufw allow OpenSSH

# SOLO en la opción B. Con túnel no hace falta abrir nada.
sudo ufw allow 'Nginx Full'

sudo ufw enable && sudo ufw status verbose
```

8 Qué protege cada capa

RIESGO	DÓNDE SE PARA
Saturación de red (DDoS)	**Cloudflare.** Un proceso de Python no puede defenderse de esto; por eso el túnel es la mejor opción.
Avalancha de peticiones	Nginx `limit_req` y, detrás, los límites por IP de la aplicación.
Quemar la CPU con renders	Tope de 2 simultáneos, 6 por IP cada 15 min, guion de 1500 caracteres y 100 segundos de video como máximo.
Accesos no autorizados	Contraseña con sesión firmada de 12 h, cookie `HttpOnly` y `SameSite=Lax`, 8 intentos cada 15 min.
Imágenes maliciosas	Se abre y verifica la imagen de verdad, no solo la extensión. Tope de 40 millones de píxeles contra bombas de descompresión.
Escapar del directorio	Los nombres de archivo se sanean y las rutas servidas se comprueban dentro del proyecto.
Robo de datos del navegador	Cabeceras `CSP`, `X-Frame-Options: DENY`, `nosniff` y `Referrer-Policy`.
Disco lleno	Los videos se borran solos a las 12 horas.

Lo que esto NO cubre

No hay cifrado de los videos en disco ni usuarios separados: quien entra, ve y descarga todo lo generado. Es una herramienta personal, no un servicio multiusuario. Si lo vas a compartir con gente ajena, eso hay que construirlo.

9 Uso diario y problemas

Comandos que vas a usar

```
sudo systemctl restart estudio        # reiniciar
journalctl -u estudio -f              # ver el registro en vivo
journalctl -u estudio --since "1 hour ago"
df -h /opt                            # espacio libre
systemd-cgtop                         # qué está consumiendo CPU
```

Actualizar

```
sudo -u estudio -H bash
cd /opt/estudio/app && git pull
/opt/estudio/venv/bin/pip install -r requirements.txt
exit
sudo systemctl restart estudio
```

Si algo falla

SÍNTOMA	CAUSA Y ARREGLO
No se encuentra FFmpeg	`sudo apt install ffmpeg`, o indica la ruta con `ESTUDIO_FFMPEG`.
No se encuentra ninguna tipografía	`sudo apt install fonts-dejavu-core`.
no hay ningún avatar medido	Falta ejecutar `medir.py` tras clonar.
Error al generar la voz	`edge-tts` abre un WebSocket saliente a Microsoft. Comprueba que el servidor tiene salida: `curl -I https://speech.platform.bing.com`
Trabajo que devuelve 404	Hay más de un `worker`. Debe ser `--workers 1`.
Subida rechazada con 413	`client_max_body_size` de Nginx por debajo de 40 MB.
Siempre pide la contraseña	`ESTUDIO_HTTPS=1` sin servir por HTTPS: el navegador descarta la cookie `Secure`.

Copia de seguridad: solo tres carpetas

`motor/avatares/`, `motor/poses_fuente/` y `motor/guiones/`. Todo lo demás —medidas, recortes, voz cacheada y videos— se regenera solo con `medir.py` y `poses.py`.